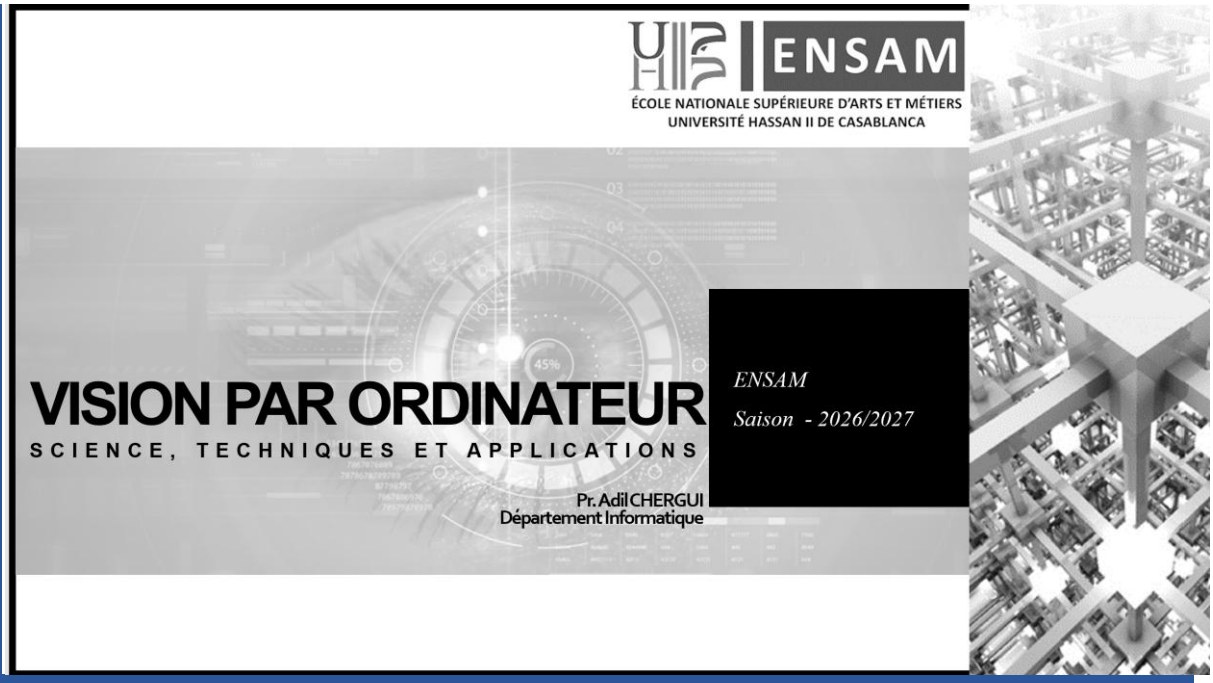


TP 1 :
Les Filtres à convolution
Python + OPENCV

Objectifs des TPs

Comprendre le principe du filtrage et son utilisation tout en préparant une image à des traitements tels que la détection de contours.

Compétences visées :

- Comprendre le principe de convolution ;
- Construire/utiliser un noyau de convolution ;
- Modifier les paramétres des kernel;
- Analyser l'effet du filtrage sur les détails ;
- Comparer plusieurs algorithme entre eux.

1. TP 1 : Filtre moyenneur

1.1. Objectif :

Comprendre le principe de la **convolution** et utiliser un **filtre moyenneur** pour réduire le bruit.
Pour une image bruitée :

- Appliquer un filtre moyenneur 3×3 ;
- Appliquer un filtre 5×5 ;
- Appliquer un filtre 15×15 ;
- Comparer les résultats ;
- Créer manuellement un noyau moyenneur ;
- Effectuer la convolution avec filter2D.

1.2. Interprétation

Pour le noyau suivant :

$$K = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Nous allons remplacer chaque pixel par la moyenne de ses voisins.
Plus le noyau est grand, plus le lissage est important, mais plus les détails sont perdus.

1.3. Implémentation

Code : exercice 1\exo1.py

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread("image_bruitee_1.jpg")

image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

blur3 = cv2.blur(image, (3, 3))
blur5 = cv2.blur(image, (5, 5))
blur8 = cv2.blur(image, (8, 8))
blur12 = cv2.blur(image, (12, 12))
blur15 = cv2.blur(image, (15, 15))

plt.figure(figsize=(12, 12))

images = [image, blur3, blur5, blur8,blur12, blur15]
titles = ["Originale.jpg", "3x3.jpg", "5x5.jpg","8x8.jpg","12x12.jpg", "15x15.jpg"]

for i in range(6):
    plt.subplot(2, 3, i+1)
    plt.imshow(images[i])
    plt.title(titles[i])
    plt.axis("off")
    image = cv2.cvtColor(images[i], cv2.COLOR_RGB2BGR)
    cv2.imwrite(titles[i],image)

plt.show()
```

1.4. Convolution manuelle

Vous pouvez construire le filtre manuellement :

```
kernel = np.ones((3, 3), np.float32) / 9

result = cv2.filter2D(image, -1, kernel)

plt.imshow(result)
plt.axis("off")
plt.show()
```

1.5. Exécution



Figure 1 : Application sur image_bruitee_1.jpg

2. TP 2 : Filtre Guassian

2.1. Objectif :

Comprendre le filtrage **gaussien** et son rôle dans la réduction du bruit.

Comparer :

- Gaussian 3×3 ;
- Gaussian 7×7 ;
- Gaussian 15×15 ;
- différentes valeurs de sigma.

2.2. Interprétation

Pour le noyau suivant :

$$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Nous allons remplacer chaque pixel par la produit de convolution de ses voisins.
Même chose : Plus le noyau est grand, plus le lissage est important, mais plus les détails sont perdus.

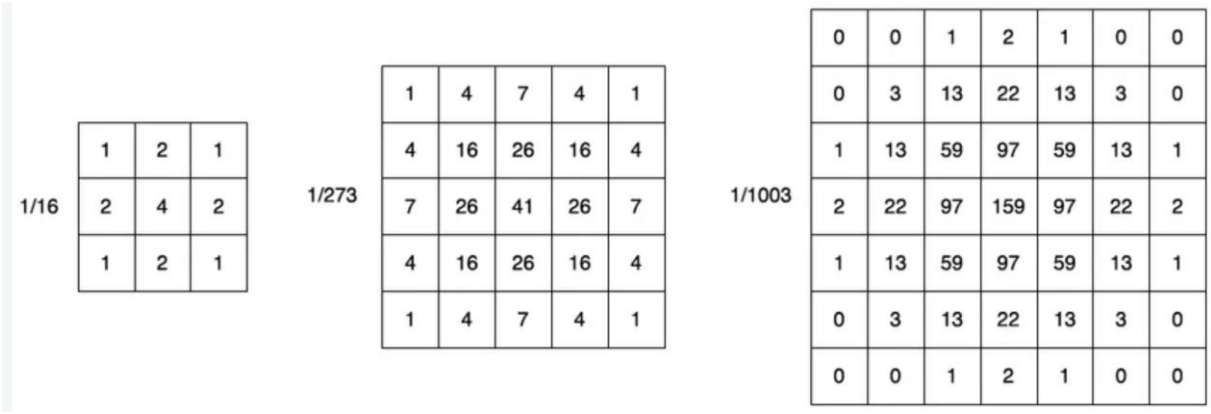


Figure 2 : Discrete approximation of the Gaussian kernels 3x3, 5x5, 7x7

2.3. Implémentation

Code : exercice 2\exo2.py

```
import cv2
import matplotlib.pyplot as plt

image = cv2.imread("image_bruitee_3.jpg")
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

g3 = cv2.GaussianBlur(image, (3, 3), 0)
g7 = cv2.GaussianBlur(image, (7, 7), 0)
g15 = cv2.GaussianBlur(image, (15, 15), 0)

plt.figure(figsize=(12, 8))

for i, (img, title) in enumerate([
    (image, "Originale.jpg"),
    (g3, "Gaussian 3x3.jpg"),
    (g7, "Gaussian 7x7.jpg"),
    (g15, "Gaussian 15x15.jpg")
]):

    plt.subplot(2, 2, i+1)
    plt.imshow(img)
    plt.title(title)
    plt.axis("off")
```

```
    image = cv2.cvtColor(img, cv2.COLOR_RGB2BGR)
    cv2.imwrite(title,image)

plt.show()
```

2.4. Exécution



Figure 3 : Application sur image_bruitee_3.jpg

2.5. Influence de sigma

Veuillez tester le changement du paramètre **sigma :**

```
g_sigma1 = cv2.GaussianBlur(image, (9, 9), 1)
g_sigma5 = cv2.GaussianBlur(image, (9, 9), 5)
g_sigma10 = cv2.GaussianBlur(image, (9, 9), 10)
```

Questions

1. Quel est l'effet de l'augmentation de sigma ?
2. Pourquoi le filtre gaussien est-il souvent utilisé avant Canny ?
3. Quelle différence avec le filtre moyenneur ?

3. TP 3 : Travail demandé Filtre médian

3.1. Objectif :

Étudier le filtre médian et son efficacité contre le **bruit impulsionnel / sel-poivre**.

Ajouter artificiellement du bruit à une image puis comparer :

- filtre moyenneur ;
- filtre gaussien ;
- filtre médian.

3.2. Interprétation

Pour le bruit impulsionnel, le filtre médian est généralement particulièrement adapté car il remplace le pixel par la **médiane de son voisinage**, ce qui limite l'influence des valeurs extrêmes.

3.3. Exécution

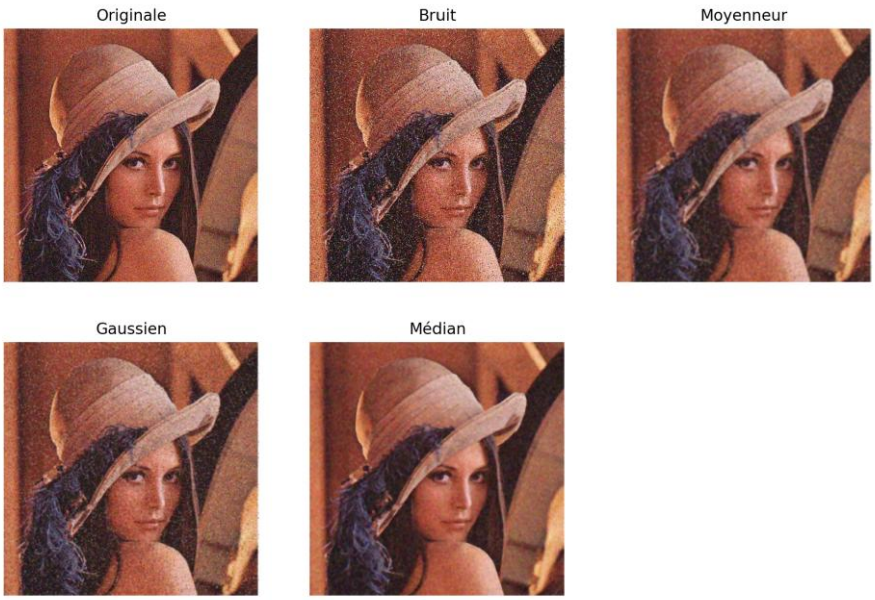


Figure 4 : Efficacité contre le bruit impulsionnel / sel-poivre

4. TP 3 : Comparaison des filtres

4.1. Objectif :

Comparer expérimentalement plusieurs filtres.
Travail demandé :
Créer trois images bruitées :

- bruit gaussien ;
- bruit sel-poivre ;
- bruit uniforme.

Tester :

- moyenneur ;
- gaussien ;
- médian ;
- bilatéral.

Calculer une mesure d'erreur entre l'image originale et l'image filtrée.

4.2. Implémentation

Le code de génération de bruit :

- bruit gaussien ;
- bruit sel-poivre ;
- bruit uniforme.

Code : exercice 4\bruit.py

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# ---------------------------------------------------
# 1. Charger l'image
# ---------------------------------------------------

image = cv2.imread("image_bruitee_4.jpg")

# Conversion BGR -> RGB pour l'affichage avec Matplotlib
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)


# ---------------------------------------------------
# 2. Bruit gaussien
# ---------------------------------------------------

def bruit_gaussien(image, moyenne=0, sigma=25):
    """
```

```
    Ajoute un bruit gaussien à l'image.
    moyenne : moyenne du bruit
    sigma   : écart-type du bruit
    """

    bruit = np.random.normal(moyenne, sigma, image.shape)

    image_bruitee = image.astype(np.float32) + bruit

    # Limiter les valeurs entre 0 et 255
    image_bruitee = np.clip(image_bruitee, 0, 255)

    return image_bruitee.astype(np.uint8)


# ------------------------------------------------
# 3. Bruit sel-poivre
# ------------------------------------------------

def bruit_sel_poivre(image, proportion=0.05):
    """
    Ajoute un bruit sel-poivre.
    proportion : proportion de pixels bruités.
    """

    image_bruitee = image.copy()

    # Nombre total de pixels
    hauteur, largeur = image.shape[:2]
    nombre_pixels = int(proportion * hauteur * largeur)

    # Pixels "sel" -> blanc
    for _ in range(nombre_pixels // 2):
        x = np.random.randint(0, hauteur)
        y = np.random.randint(0, largeur)
        image_bruitee[x, y] = 255

    # Pixels "poivre" -> noir
    for _ in range(nombre_pixels // 2):
        x = np.random.randint(0, hauteur)
        y = np.random.randint(0, largeur)
        image_bruitee[x, y] = 0

    return image_bruitee


# ------------------------------------------------
# 4. Bruit uniforme
# ------------------------------------------------

def bruit_uniforme(image, minimum=-30, maximum=30):
    """
    Ajoute un bruit uniforme à l'image.
    """

    bruit = np.random.uniform(
```

```
        minimum,
        maximum,
        image.shape
    )

    image_bruitee = image.astype(np.float32) + bruit

    # Limiter les valeurs entre 0 et 255
    image_bruitee = np.clip(image_bruitee, 0, 255)

    return image_bruitee.astype(np.uint8)


# ------------------------------------------------
# 5. Génération des trois images bruitées
# ------------------------------------------------

image_gaussien = bruit_gaussien(image)
image_sel_poivre = bruit_sel_poivre(image)
image_uniforme = bruit_uniforme(image)


# ------------------------------------------------
# 6. Affichage
# ------------------------------------------------

plt.figure(figsize=(12, 8))

plt.subplot(2, 2, 1)
plt.imshow(image_rgb)
plt.title("Image originale")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(cv2.cvtColor(image_gaussien, cv2.COLOR_BGR2RGB))
plt.title("Bruit gaussien")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(cv2.cvtColor(image_sel_poivre, cv2.COLOR_BGR2RGB))
plt.title("Bruit sel-poivre")
plt.axis("off")

plt.subplot(2, 2, 4)
plt.imshow(cv2.cvtColor(image_uniforme, cv2.COLOR_BGR2RGB))
plt.title("Bruit uniforme")
plt.axis("off")

plt.tight_layout()
plt.show()


# ------------------------------------------------
# 7. Sauvegarde des images
# ------------------------------------------------
```

```
cv2.imwrite("image_bruit_gaussien.jpg", image_gaussien)
cv2.imwrite("image_bruit_sel_poivre.jpg", image_sel_poivre)
cv2.imwrite("image_bruit_uniforme.jpg", image_uniforme)
```

Le code de calcul de différence avec MSE :

Code : exercice 4\exo4.py

```
import cv2
import numpy as np

original = cv2.imread("image_bruitee_2.jpg")

gray = cv2.cvtColor(original, cv2.COLOR_BGR2GRAY)

mean = cv2.blur(gray, (5,5))
gaussian = cv2.GaussianBlur(gray, (5,5), 0)
median = cv2.medianBlur(gray, 5)
bilateral = cv2.bilateralFilter(gray, 9, 75, 75)

def mse(img1, img2):
    return np.mean((img1.astype(float) - img2.astype(float)) ** 2)

print("MSE moyenneur :", mse(gray, mean))
print("MSE gaussien :", mse(gray, gaussian))
print("MSE médian :", mse(gray, median))
print("MSE bilatéral :", mse(gray, bilateral))
```

4.3. Interprétation

Resultats d'execution donne :

```
MSE moyenneur : 322.09620006173384
MSE gaussien : 261.74951494840815
MSE médian : 332.20310984213774
MSE bilatéral : 260.14361385483727
```

Question importante

Une MSE plus faible signifie-t-elle toujours que l'image est visuellement meilleure ?

Non. Une métrique numérique doit être interprétée selon l'objectif du traitement.

5. TP 5 : Gradient et filtre de Sobel

5.1. Objectif :

Comprendre les dérivées d'image et détecter les contours avec Sobel.

Calculer :

- Gradient horizontal ;
- Gradient vertical ;
- Magnitude du gradient ;

Comparer différents noyaux Sobel.

5.2. Interprétation

Pour la formule suivante :

$$G = \sqrt{G_x^2 + G_y^2}$$

Le filtre matriciel de taille 3x3 est généré :

-1	0	+1
-2	0	+2
-1	0	+1

Gx

+1	+2	+1
0	0	0
-1	-2	-1

Gy

Nous allons remplacer chaque pixel par la moyenne de ses voisins.
Plus le noyau est grand, plus le lissage est important, mais plus les détails sont perdus.

5.3. Implémentation

Code : exercice 5\exo5.py

Vous pouvez construire le filtre manuellement :

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread("cells3.jpg", cv2.IMREAD_GRAYSCALE)
```

```python
sobel_x = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(image, cv2.CV_64F, 0, 1, ksize=3)

magnitude = cv2.magnitude(
    sobel_x.astype(np.float32),
    sobel_y.astype(np.float32)
)

sobel_x = cv2.convertScaleAbs(sobel_x)
sobel_y = cv2.convertScaleAbs(magnitude)

plt.figure(figsize=(16,4))

plt.subplot(1,4,1)
plt.imshow(image, cmap="gray")
plt.title("Originale")

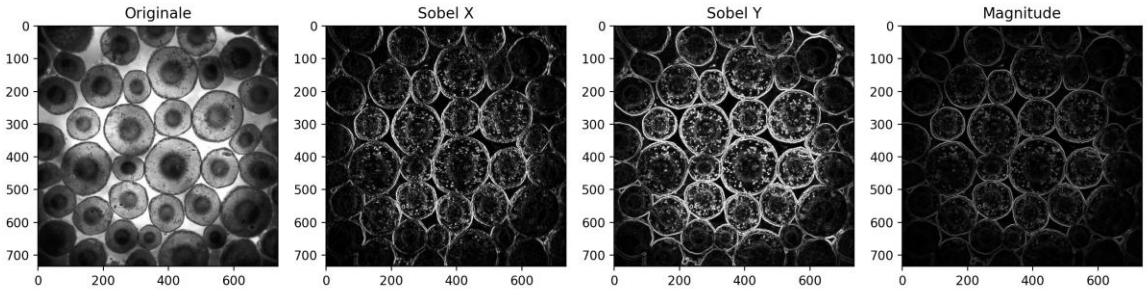
plt.subplot(1,4,2)
plt.imshow(sobel_x, cmap="gray")
plt.title("Sobel X")

plt.subplot(1,4,3)
plt.imshow(sobel_y, cmap="gray")
plt.title("Sobel Y")

plt.subplot(1,4,4)
plt.imshow(magnitude, cmap="gray")
plt.title("Magnitude")


plt.show()
```

5.4. Exécution



6. TP 6 : Laplacien et amélioration d'image

6.1. Objectif :

Comprendre le Laplacien et son utilisation pour détecter les variations rapides d'intensité et accentuer les contours.

6.2. Implémentation

Code : exercice 6\exo6.py

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread("coins.jpg", cv2.IMREAD_GRAYSCALE)

laplacian = cv2.Laplacian(image, cv2.CV_64F)

laplacian_abs = cv2.convertScaleAbs(laplacian)

plt.figure(figsize=(10,4))

plt.subplot(1,2,1)
plt.imshow(image, cmap="gray")
plt.title("Originale")

plt.subplot(1,2,2)
plt.imshow(laplacian_abs, cmap="gray")
plt.title("Laplacien")

plt.show()
```

Ou Manuellement :

```
kernel = np.array([
    [0, -1, 0],
    [-1, 5, -1],
    [0, -1, 0]
])

sharp = cv2.filter2D(image, -1, kernel)

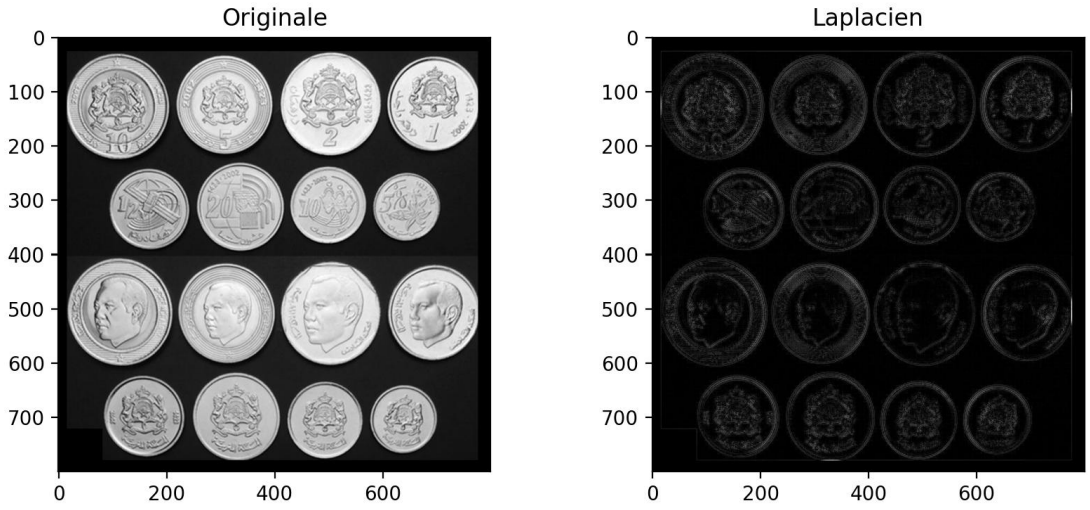
plt.imshow(sharp, cmap="gray")
plt.axis("off")
plt.show()
```

6.3. Interpretation

Questions

- Testez d'appliquer un lissage avant le **Laplacien** ?
- Pourquoi le **Laplacien** est-il sensible au bruit ?

6.4. Exécution



7. TP 7 : Détection de contours avec Canny

7.1. Objectif :

Comprendre l'algorithme de Canny.

Étapes

- réduction du bruit ;
- calcul du gradient ;
- suppression des non-maxima ;
- double seuillage ;
- hystérésis.

7.2. Fonctionnement

Les étapes de l'algorithme

- **Le lissage (Flou gaussien) :** L'image subit un flou pour effacer le bruit visuel (les petits points parasites).
- **Le calcul du gradient :** L'algorithme calcule la variation de luminosité (le gradient) pour repérer les zones de transition entre le clair et le sombre.

  Exemple

$$h = \frac{1}{159}\begin{bmatrix} 2 & 4 & 5 & 4 & 2 \\ 4 & 9 & 12 & 9 & 4 \\ 5 & 12 & 15 & 12 & 5 \\ 4 & 9 & 12 & 9 & 4 \\ 2 & 4 & 5 & 4 & 2 \end{bmatrix}$$

- **La suppression des non-maxima :** L'image est affinée pour garder uniquement les pixels les plus intenses et obtenir des traits fins.
- **Le double seuillage :** Deux seuils (haut et bas) sont fixés. Les pixels au-dessus du seuil haut forment les vrais contours, ceux en dessous sont rejetés, et ceux entre les deux sont gardés s'ils touchent un contour valide.

7.3. Implémentation

Code : exercice 7\exo7.py

```
import cv2
import matplotlib.pyplot as plt

image = cv2.imread("cells.jpg", cv2.IMREAD_GRAYSCALE)

blur = cv2.GaussianBlur(image, (5,5), 0)

edges1 = cv2.Canny(blur, 50, 150)
edges2 = cv2.Canny(blur, 100, 200)
edges3 = cv2.Canny(blur, 150, 250)

plt.figure(figsize=(12,8))

for i, (img, title) in enumerate([
    (image, "Originale"),
    (edges1, "50-150"),
    (edges2, "100-200"),
    (edges3, "150-250")
]):

    plt.subplot(2,2,i+1)
    plt.imshow(img, cmap="gray")
```

```
    plt.title(title)
    plt.axis("off")

plt.show()
```

7.4. Interpretation

Questions

1. Que se passe-t-il si les seuils sont trop faibles ?
2. Que se passe-t-il s'ils sont trop élevés ?
3. Pourquoi effectuer un Gaussian Blur avant **Canny** ?

7.5. Exécution

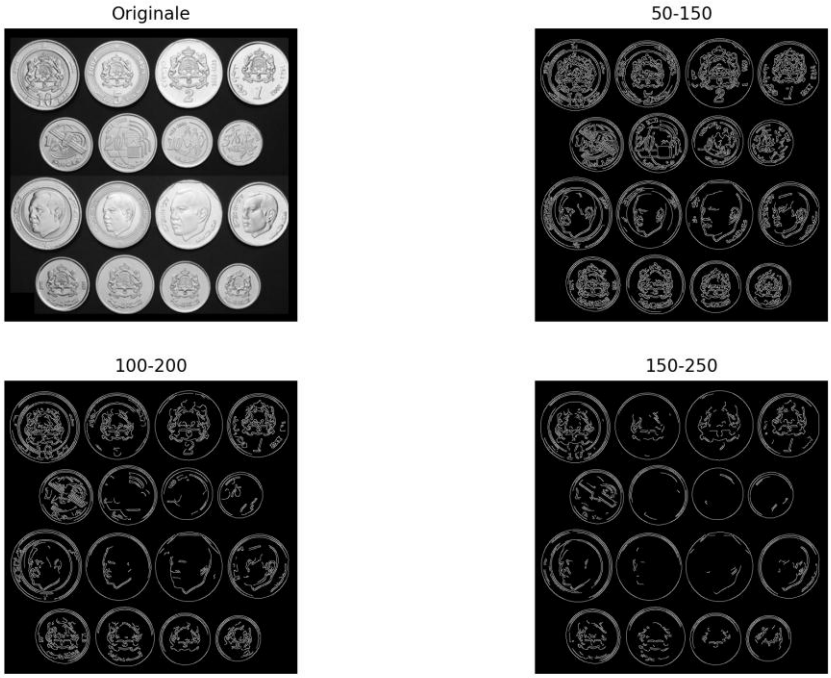


Figure 5 : Execution sur coins.jpg

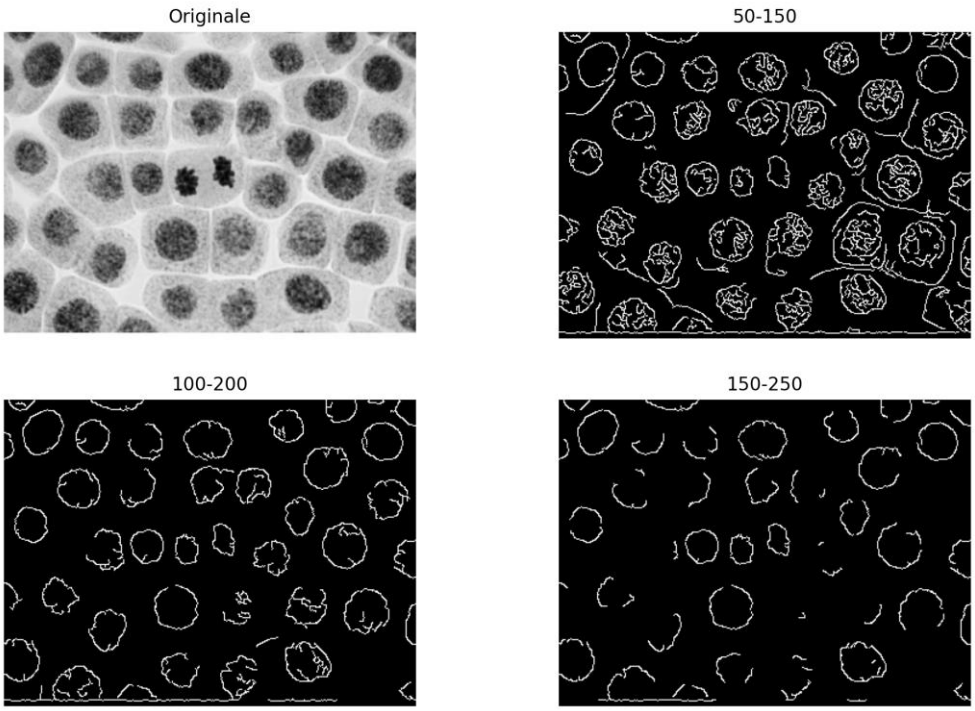


Figure 6 : Execution sur cellss.jpg

8. Mini-projet

8.1. Objectif :

Construire un pipeline complet :
Image → Grayscale → Réduction du bruit → Filtrage → Gradient → Canny → Contours

8.2. Travail demandé

Développer un programme permettant à l'utilisateur de choisir :

- le filtre ;
- la taille du noyau ;
- les seuils Canny.

Avec interfaçage interactive à l'aide d'OpenCV

Il sera aussi demandé de tester :

- Gaussian + Canny

- Median + Canny
- Bilateral + Canny